O QUE É:

É um meio padronizado de comunicação entre duas aplicações.

PRA QUE SERVE:

Utilizada principalmente em aplicações web para fazer duas aplicações conversarem de forma padronizada.

COMO FUNCIONA:

Através de um padrão de requisições (**requests**) e respostas (**responses**).

O protocolo **HTTP** (*Hypertext Transfer Protocol*) funciona no modelo **cliente-servidor**, onde a comunicação acontece por meio de um ciclo de duas partes: a **Request** (requisição enviada pelo cliente) e a **Response** (resposta devolvida pelo servidor). [1, 2]

É uma comunicação baseada em "pedido e resposta": o cliente pede algo, o servidor processa e entrega. [1, 2]

---

1. O Request (Requisição)

É a mensagem enviada pelo **cliente** (como o seu navegador, um aplicativo de celular ou um script) para o servidor pedindo alguma ação ou recurso. [1, 2]

Uma requisição HTTP é composta por:

- **Linha de Requisição (Request Line):**
  - **Verbo HTTP (Método):** Indica o *tipo* de ação que o cliente quer realizar.
    Os principais são:
    - `GET`: Para buscar/obter dados do servidor.
    - `POST`: Para enviar novos dados para o servidor (ex: enviar formulário).
    - `PUT` / `PATCH`: Para atualizar dados existentes.
    - `DELETE`: Para apagar dados.
  - **URL (ou URI):** O endereço do recurso que está sendo acessado.
  - **Versão do Protocolo:** Exemplo: `HTTP/1.1`, `HTTP/2` ou `HTTP/3`. [1, 2, 3, 4, 5, 6]
- **Cabeçalhos (Request Headers):** Metadados que dão contexto sobre a requisição (ex: qual o navegador usado, quais idiomas o cliente aceita, formato de dados esperado, tokens de autenticação). [1]

- **Corpo (Request Body) *(Opcional)*:** Usado em métodos como `POST` ou `PUT`, onde você precisa enviar dados reais para o servidor (por exemplo, o JSON com o seu login e senha). [1, 2, 3]

---

2. O Response (Resposta)

É a mensagem enviada de volta pelo **servidor** para o cliente logo após processar a requisição. [1, 2]

Uma resposta HTTP é composta por:

- **Linha de Status (Status Line):**
  - **Versão do HTTP:** Exemplo: `HTTP/1.1`.
  - **Código de Status (Status Code):** Um número de 3 dígitos que diz se deu certo, se deu erro ou se precisa de algo mais:
    - **1xx (Informativo):** A requisição foi recebida e o processo continua.
    - **2xx (Sucesso):** Deu tudo certo (ex: `200 OK`, `201 Created`).
    - **3xx (Redirecionamento):** O recurso mudou de lugar (ex: `301 Moved Permanently`).
    - **4xx (Erro do Cliente):** O pedido foi feito errado ou não foi encontrado (ex: `404 Not Found`, `401 Unauthorized`).
    - **5xx (Erro do Servidor):** O servidor teve uma pane ao tentar processar (ex: `500 Internal Server Error`). [1, 2]
- **Cabeçalhos (Response Headers):** Informações sobre a resposta (ex: tipo do conteúdo retornado como `Content-Type: application/json`, data, tamanho do arquivo). [1]
- **Corpo (Response Body) *(Opcional)*:** O dado que foi pedido. Pode ser o código HTML de uma página web, um arquivo JSON de uma API, uma imagem ou um texto. [1]

Exemplo prático do fluxo

1. **Você** abre o navegador e digita um site (ou clica em um botão).
2. O **Navegador (Cliente)** monta um **Request**:
   - Método: `GET`
   - URL: `https://exemplo.com`
   - Headers: `User-Agent: Chrome...`
3. O **Servidor** recebe esse pedido, busca as informações no banco de dados e monta um **Response**:

- Status Code: `200 OK`
- Headers: `Content-Type: application/json`
- Body: `{"nome": "João", "idade": 30}`

4. O **Navegador** recebe o response, lê o conteúdo e exibe a tela ou processa os dados para você. [1]

Para se aprofundar na documentação oficial dos métodos e estrutura, consulte o MDN Web Docs - Métodos HTTP. [1]